VitalFlow V1 — AI Scribe Architecture

Design + scaffold for the AI scribe MVP. Produces SOAP drafts + ICD-10/CPT suggestions from audio or a pasted transcript. Output stays advisory — a physician must review, accept, edit, and sign. The draft flows through the existing encounter-notes versioning system (docs/encounter-workspace.md) so amendments and audit trails work unchanged.

See also:

- docs/permissions-matrix.md — `ai:invoke` required for every call
- docs/audit-logging.md — semantic audit events
- docs/clinical-domain.md — `DiagnosisAssignment` target for accepted codes

---

1. End-to-end architecture

```
┌──────────────────┐   1. Create scribe session
│ Physician, nurse │ ──────────────────────────▶ ┌──────────────────┐
│ (encounter open) │                             │ scribe_sessions row │
└──────────────────┘                             │ status=pending   │
        │                                        └──────────────────┘
        │ 2. Upload audio                                │
        │ OR paste transcript                            ▼
        ▼
┌──────────────────┐   3a. audio → Gemini Flash   ┌──────────────────┐
│ Supabase Storage │ ──────────────────────────▶ │ transcribe step  │
│ bucket: scribe-raw │       (multimodal, cheap)  │ ai_requests row (1) │
└──────────────────┘                             │ status=running   │
                                                 └──────────────────┘
                                                        │ segments[] w/ ts
                                                        ▼
                                                 ┌──────────────────┐
                                                 │ ai_scribe_transcript │
                                                 │ _segments (N rows) │
                                                 │ each with ts + id │
                                                 └──────────────────┘
                                                        │ 4. SOAP synth
                                                        ▼
                                        ┌──────────────────────────────┐
                                        │ generate step                │
                                        │ ai_requests row (2)          │
                                        │ model: claude-opus-4-7 (default) │
                                        │ input: segments + patient context │
                                        │ output: structured JSON +    │
                                        │   per-section trace segment ids │
                                        └──────────────────────────────┘
                                                │
                                                ▼
                                        ┌──────────────────────────────┐
                                        │ ai_completions row w/        │
                                        │   structured SOAP + trace refs │
                                        └──────────────────────────────┘
                                                │ 5. Code suggestion
                                                ▼
                                        ┌──────────────────────────────┐
                                        │ suggest-codes step           │
                                        │ ai_requests row (3)          │
                                        │ model: gemini-flash (cheap)  │
                                        │ output: { icd10[], cpt[] } w/ │
                                        │   confidence + trace         │
                                        └──────────────────────────────┘
                                                │
                                                ▼
                                        ┌──────────────────────────────┐
                                        │ ai_scribe_code_suggestions (N) │
                                        └──────────────────────────────┘
                                                │ 6. Physician review
                                                ▼
                                        ┌──────────────────────────────┐
                                        │ /encounters/[id] — draft appears │
                                        │ w/ Accept into note action   │
```

```
                                        └──────────────────────────────────────┘

Accept into note:
  - inserts encounter_notes row, status='draft', ai_assisted=true, ai_request_id set
  - physician then edits/signs via existing Slice 3 flow
  - ai.draft_accepted audit event emitted
```

Key invariants visible in the diagram:

- Three `ai_requests` rows per scribe session — transcribe / generate / suggest-codes — each independently observable, retryable, and audit-logged.
- Everything ties back to one `ai_scribe_sessions.id` via `ai_requests.correlation_id`.
- The physician accepts the draft into an `encounter_notes` row; signing remains a manual, separate action.

---

2. Services involved

Service	Lives in	Responsibility
`ScribeSessionService`	`@vitalflow/ai/scribe`	Orchestrate the 3-step pipeline; own the `ai_scribe_sessions` lifecycle
`TranscriptionService`	`@vitalflow/ai/scribe`	Audio → text + timestamped segments. Uses Gemini Flash multimodal
`SoapDraftService`	`@vitalflow/ai/scribe`	Transcript segments + patient context → structured SOAP JSON + trace refs. Uses Claude Opus
`CodeSuggestionService`	`@vitalflow/ai/scribe`	SOAP + transcript → ICD-10 + CPT suggestions with confidence. Uses Gemini Flash
`AIProvider` (existing)	`@vitalflow/ai/providers`	Anthropic / OpenAI / Google adapters; new `google.ts` added
`AuditService` (existing)	`@vitalflow/auth/audit`	`logEvent("ai.draft_generated"
`StorageService`	`@vitalflow/ai/scribe` (thin wrapper)	Supabase Storage bucket `scribe-raw` — audio files, auto-deleted after 90 days
`SafetyGuardrails` (existing)	`@vitalflow/ai/guardrails`	Redact PHI before prompt, block forbidden content

Models per step (V1):

- **Transcription:** `gemini-2.0-flash` — cheap, multimodal, ~$0.075 / 1M input tokens at audio rates.
- **SOAP synthesis:** `claude-opus-4-7` — reasoning quality for clinical narrative.
- **Code suggestion:** `gemini-2.0-flash` — deterministic classification, cost-sensitive.

Swappable via `ai_requests.model` + env-driven default.

3. Job lifecycle

3.1 `ai_scribe_sessions.status` state machine

```
          ┌──── pending ────┐
          │                 │ submit audio/transcript
          ▼                 ▼
    cancelled           transcribing
                             │
                             ▼
                        generating
                             │
                             ▼
                     suggesting_codes
                             │
                             ▼
                      awaiting_review ──── physician accepts ──▶ accepted
                             │                                      │
                             │ physician rejects                    │ (terminal)
                             ▼
                        rejected (terminal)

  any state → failed (terminal; error_message populated)
```

A step failure (e.g., transcription times out) does **not** fail the whole session unless the user abandons it — they can retry the failing step, or fall back to pasting a transcript and skip transcription.

3.2 Timing + budgets

Step	Typical	Timeout	Cost target (30-min visit)
Transcription	10–40s	90s	~$0.01
SOAP synthesis	5–15s	60s	~$0.08
Code suggestion	2–5s	30s	~$0.005
Total	~15–60s wall	—	**~$0.10**

Wall-clock total is within Vercel's default 60s function timeout if we run steps sequentially inline. For longer audio, V2 pushes steps into Supabase Edge Functions with DB-triggered background execution.

3.3 Execution strategy (V1)

Next.js Route Handler receives the request and does the work inline. UI polls the `GET /sessions/:id` endpoint every 2s. Status transitions come from the same Route Handler updating `ai_scribe_sessions.status` after each step completes.

Pros: no worker infra, fits MVP. Cons: tied to single HTTP lifecycle; >60s audio fails (accept the limit). Document and move on.

---

4. Input / output contracts

4.1 Create session

`POST /api/v1/ai/scribe/sessions`

```
// Request
{
  "encounterId": "uuid",
  "source": "audio_upload" | "transcript_paste" | "stream",
  "model_overrides": {                  // optional
    "transcribe": "gemini-2.0-flash",
    "generate":   "claude-opus-4-7",
    "codes":      "gemini-2.0-flash"
  }
}
```

```
// Response
{
  "sessionId": "uuid",
  "status": "pending",
  "uploadUrl": "https://…"              // null if source=transcript_paste
}
```

4.2 Submit content

`POST /api/v1/ai/scribe/sessions/:id/transcript`

```
// Request — audio uploaded via uploadUrl, now trigger processing
{
  "storagePath": "scribe-raw/…"         // returned from upload
}
// OR
{
  "text": "... raw transcript ..."      // for transcript_paste
}
```

4.3 Session GET — polling endpoint

`GET /api/v1/ai/scribe/sessions/:id`

```
{
  "sessionId": "uuid",
  "status": "generating",
  "steps": {
    "transcribe": { "status": "completed", "latency_ms": 18432, "request_id": "…" },
    "generate":   { "status": "running",   "started_at": "2026-04-20T…" },
    "codes":      { "status": "pending" }
  },
  "draft": null,                          // populated once generate completes
  "codes": [],                            // populated once suggest completes
  "transcript": {
    "segments": [{ "id": "uuid", "startMs": 0, "endMs": 8400, "text": "..." }]
  }
}
```

4.4 Draft payload (after generate)

```
{
  "draft": {
    "subjective": {
      "text": "Chief complaint of sore throat for 3 days…",
      "segmentIds": ["…", "…"],           // trace refs
      "confidence": 0.87
    },
    "objective":  { "text": "…", "segmentIds": [...], "confidence": 0.79 },
    "assessment": { "text": "…", "segmentIds": [...], "confidence": 0.72 },
    "plan":       { "text": "…", "segmentIds": [...], "confidence": 0.81 },
    "warnings": [
      "Medication 'metformin' mentioned but not in current med list — verify"
    ]
  }
}
```

4.5 Code suggestions

```
{
  "codes": [
    {
      "id": "uuid",
      "codeSystem": "icd10-cm",
      "code": "J02.9",
      "description": "Acute pharyngitis, unspecified",
      "confidence": 0.84,
      "segmentIds": ["…"],
      "rank": 1,
      "type": "diagnosis"
    },
    {
      "id": "uuid",
      "codeSystem": "cpt",
      "code": "99213",
      "description": "Office visit, established patient, low complexity",
      "confidence": 0.71,
      "segmentIds": [],
      "rank": 1,
      "type": "procedure"
    }
  ]
}
```

4.6 Accept

`POST /api/v1/ai/scribe/sessions/:id/accept`

```
// Request
{
  "acceptedCodes": ["code-suggestion-uuid", ...],
  "editedDraft": {                        // optional — if physician inline-edits
    "subjective": "…", "objective": "…", "assessment": "…", "plan": "…"
  }
}
```

Response: the newly-inserted `encounter_notes.id` (status= `draft` , `ai_assisted=true`).

4.7 Reject

`POST /api/v1/ai/scribe/sessions/:id/reject`

```
{
  "reason": "Hallucinated medication",
  "correction": "…"                      // optional — what should have been said
}
```

Writes `ai_feedback` row. Session status → `rejected`.

---

5. Safety rules

1. **No auto-sign.** The only path to `encounter_notes.status='signed'` is the existing `signNote` server action driven by a human. The scribe accept action writes `status='draft'` and returns. Enforced at both app layer and by physically separate endpoints.
2. **`ai:invoke` gate.** Every endpoint requires the `ai:invoke` permission. Only clinical roles (`physician`, `nurse_ma`, `practice_owner`) carry it in V1. Schedulers, billers, patients never invoke.
3. **Encounter tenancy.** Session `tenant_id` must match the encounter's tenant. RLS enforces this on reads; app code on writes.
4. **Impersonation block.** `ai.draft_accepted` requires `notes:amend`-equivalent semantics (it writes clinical content on behalf of a user). The action refuses while `isImpersonating` is true. A super-admin cannot silently inject AI content into a practice.
5. **PHI redaction before prompt.** The transcript + context sent to the LLM is the patient's own data; that's fine *within* the HIPAA-covered relationship. But we never send **other** patients' data. `redacted_context.jsonb` on `ai_requests` records what we sent (minus the patient's own PHI, which is referenced by id).
6. **No model fine-tuning on PHI without BAA.** Providers used in V1 (Anthropic, Google) both offer BAA-compliant endpoints. Disabled for prompt retention if BAA'd; the `ai_requests.prompt_hash` is still stored for traceability.
7. **Hallucination warnings.** The SOAP synth prompt instructs the model to emit a `warnings[]` array when it references anything not present in the transcript or patient context. The UI surfaces warnings prominently before accept.
8. **Confidence floor.** SOAP sections below a configurable threshold (default 0.5) are rendered with a "Low confidence — review carefully" badge. Codes below 0.5 are hidden by default.
9. **Transcript retention.** Raw audio auto-deleted 90 days after session creation (Storage lifecycle rule). Transcript text stays with the note for auditability.
10. **Never overwrite a signed note.** If an encounter already has a signed note, the scribe can still generate a draft but the accept action will refuse — the physician must amend the signed note manually via the existing amendment flow.

---

6. Confidence metadata design

Every AI output surfaces a numeric confidence ∈ [0, 1] and a provenance breadcrumb.

6.1 Computation

- **SOAP synthesis:** Claude doesn't emit logprobs; we elicit a self-assessment in the output schema (`"confidence": 0.xx`). We also compute a **grounding score** = (tokens matched to transcript segments) / (total output tokens). Both stored.
- **Code suggestions:** Gemini's structured output includes `confidence` in the schema. Cross-check: if the code's description doesn't pattern-match anything in the SOAP text, confidence is penalized 20%.

6.2 Shape

```
interface Confidence {
  model_self: number;      // 0..1, reported by the model
  grounding: number;       // 0..1, computed from segment coverage
  combined: number;        // harmonic mean, displayed to user
}
```

Stored on:

- `ai_completions.structured_output.confidence` (per section)
- `ai_scribe_code_suggestions.confidence` (per code)

6.3 Rendering rules

- `combined >= 0.8`: no badge, default styling.
- `0.5 <= combined < 0.8`: ⚠ amber "Review" badge.
- `combined < 0.5`: 🚫 red "Low confidence" badge; for codes, hidden behind a "Show low-confidence suggestions" toggle.

---

7. Trace reference design

Every AI-generated span references the transcript segment(s) it came from.

7.1 Segment table

```
ai_scribe_transcript_segments
├── id uuid PK
├── tenant_id
├── session_id → ai_scribe_sessions.id
├── sequence_index smallint       (0, 1, 2, …)
├── start_ms int                  (nullable for pasted transcripts)
├── end_ms int                    (nullable for pasted transcripts)
├── speaker text                  (nullable — diarization not in V1)
├── text text                     (the chunk)
└── created_at
```

A pasted transcript is chunked into ~200-word segments by newline/sentence heuristics; audio transcription produces natural segments from the ASR output.

7.2 Trace reference format

Each AI output section carries a `segmentIds: uuid[]` referencing the segments it synthesized from. The UI renders clickable markers; clicking highlights the referenced segments in the transcript panel.

For free-text inside a section, a fine-grained variant (character-offset ranges inside each segment) is possible but deferred — V1 ties references at the section level.

7.3 Integrity

- Server-side validation: any `segmentIds` returned by the model must reference segments belonging to this session. If not, the segment is dropped and a `ai.hallucinated_trace` warning is appended.
- This also defends against prompt-injection attacks where a malicious transcript instructs the model to reference a bogus ID.

---

8. Failure handling

Failure	Handling
Audio upload > size limit	Reject at API gateway (50 MB max). UI shows helpful message.
Transcription timeout	Mark transcribe step `failed`. User can retry, or switch to `transcript_paste`.
Transcription partial result	Save what we got; segments flagged `partial=true`. SOAP synth still runs.
SOAP synth fails	Mark step failed. Retry button in UI. Transcript remains.
SOAP synth returns invalid JSON	Log raw output; use LLM-repair prompt once; if still invalid, fail step + show raw text to user as a fallback.
Code suggestion fails	Non-fatal — draft still usable; user adds codes manually later. Session stays `awaiting_review`.

Provider rate limit (429)	Exponential backoff up to 3 retries with 2/4/8s delays. Give up and fail step.
Provider outage (5xx)	Same retry policy, but after 3 failures, suggest switching model via `model_overrides`.
DB write fails mid-pipeline	Next poll re-reads state; steps are idempotent by session+step pair.
PHI redaction rejects input	Mark session `blocked` (ai_requests.safety_verdict='block'). Audit event `ai.invocation_blocked`.
User cancels session mid-pipeline	Set status= `cancelled`; running request is abandoned (no cost recovery).
Accept races with amend of a signed note	Accept action re-checks note status; if signed, returns 409 and asks user to amend manually.

---

9. Audit events

All emitted via `@vitalflow/auth/audit`. Row-level DB triggers on `ai_scribe_sessions` / `ai_scribe_transcript_segments` / `ai_scribe_code_suggestions` also fire — semantic events enrich the trigger data with intent.

Event type	When	Details
`ai.scribe_session_created`	Session INSERT via Create endpoint	`encounter_id`, `source`, `models`
`ai.transcript_submitted`	After transcript text or audio is saved	`session_id`, `source`, `byte_size`
`ai.draft_generated`	SOAP synth completes	`completion_id`, `model`, `tokens_in`, `tokens_out`, `latency_ms`, `confidence_avg`
`ai.codes_suggested`	Code suggestion completes	`completion_id`, `num_codes`, `model`
`ai.draft_accepted`	User accepts into encounter_notes	`session_id`, `note_id`, `accepted_code_count`, `edits_bytes`
`ai.draft_rejected`	User rejects	`session_id`, `reason`, `correction_present`
`ai.invocation_blocked`	Safety guardrail blocks	`session_id`, `step`, `safety_reason`
`ai.hallucinated_trace`	Server drops invalid segmentIds	`session_id`, `step`, `dropped_count`

---

10. Acceptance criteria

10.1 Create session

- POST with a valid `encounterId` returns a session and a signed upload URL (if `source=audio_upload`).

- User without `ai:invoke` gets 403.
- Encounter in another tenant → 404.
- Encounter already has a signed current note → session allowed, but `/accept` later returns 409.

10.2 Transcription

- Audio file under 50 MB, under 60 min: completes within 90s on typical connection.
- Produces ≥1 segment with non-empty text.
- Segments stored with `sequence_index` 0..N and valid `start_ms <= end_ms`.
- Failure leaves session in `failed` with non-null `error_message`.

10.3 SOAP generation

- Each of subjective / objective / assessment / plan produced (can be "not documented" placeholder).
- `segmentIds` all reference segments belonging to this session.
- `confidence` populated for every section.
- `warnings[]` present in response shape (empty array if none).

10.4 Code suggestions

- Returns array of 0..N suggestions with `code`, `description`, `codeSystem`, `confidence`, `rank`.
- ICD-10 codes match the regex `^[A-Z][0-9]{2}(\.[0-9A-Z]{1,4})?$`.
- CPT codes match `^\d{5}$`.

10.5 Polling

- `GET /sessions/:id` reflects current status within 1s of transition.
- Completed steps expose their `request_id` so users can debug latency/cost.

10.6 Accept

- Inserts one row into `encounter_notes` with `status='draft'`, `ai_assisted=true`, `ai_request_id` = generate step id.
- If `acceptedCodes[]` non-empty, inserts matching `diagnosis_assignments` (for codeSystem= `icd10-cm`).
- Returns the created `note_id`. Session → `accepted`.
- Refuses if encounter has a signed current note (409).

10.7 Reject

- Writes `ai_feedback` row.
- Session → `rejected`.

- Audit event fires.

10.8 No auto-sign

- No path in any endpoint writes `status='signed'` on `encounter_notes`.
- Test covers: "after accept, fetch note — status MUST be draft".

10.9 Cost + observability

- Each of the 3 steps appears as an `ai_requests` row with token counts + latency.
- Session summary returns `total_cost_micros_usd` and `total_latency_ms`.

---

11. Rollout plan

This PR (design + scaffold):

- Design doc (this file) + PDF.
- Migration: `ai_scribe_sessions`, `ai_scribe_transcript_segments`, `ai_scribe_code_suggestions` tables, RLS, audit triggers. `google` value added to `ai_provider` enum.
- TypeScript types in `@vitalflow/types` (extends AI module).
- Service interfaces in `@vitalflow/ai/scribe`.
- API route stubs under `app/api/v1/ai/scribe/*` returning 501 with documented response shapes.

Next PR (first working prototype):

- Gemini provider class (`packages/ai/src/providers/google.ts`).
- `TranscriptionService` implementation (audio → Gemini → segments).
- `SoapDraftService` implementation (Claude call + JSON-schema response).
- `CodeSuggestionService` implementation (Gemini).
- Wire all three into the Route Handlers.
- Simple audio upload UI in the encounter workspace.

Later:

- Streaming transcript (`source='stream'`).
- Draft review UI with transcript playback + segment highlighting.
- Supabase Edge Function for background processing of long audio.
- Cost dashboard in `/admin`.

---

12. Not V1

- **Real-time streaming** — the `source='stream'` enum value is reserved but the endpoint refuses until the WebSocket transport lands.
- **Multi-speaker diarization** — the `speaker` column is there, populated null.
- **Fine-grained trace refs** at character level inside each segment — section-level refs are what V1 renders.
- **Model routing** beyond per-step defaults — no rule-based cost/quality routing yet.
- **Patient-facing AI disclosures** — the `/my` surface doesn't surface AI drafts yet; that needs consent workflow.
- **Continuous fine-tuning** on `ai_feedback` — the data is captured; pipeline is V2+.